关于构建SKill的完整指南

适用于

Claude

Claude

目录

简介 3 基础知识 4 规划与设计 7 测试与迭代 14 发布与共享
18 模式与故障排除 21 资源与参考 28

简介

技能是一组指令——以一个简单文件夹的形式打包——用于教导 Claude 如何处理特定任务或工作流。技能是为你的具体需求定制 Claude 的最强大方式之一。与其在每次对话中重复说明你的偏好、流程和领域专长，不如通过技能只需教一次 Claude，就能在每次使用时受益。

当你拥有可重复的工作流时，技能非常有用：根据规格生成前端设计、以一致的方法开展研究、创建遵循团队风格指南的文档或协调多步骤流程。它们能与 Claude 的内建能力（如代码执行和文档创建）很好地协同工作。对于构建 MCP 集成的人来说，技能提供了另一层强大功能，帮助将原始工具访问转变为可靠、优化的工作流。

本指南涵盖了从规划与结构到测试与分发，构建高效技能所需了解的一切。无论你是为自己、团队，还是为社区构建技能，都能在文中找到实用模式和真实案例。

你将学到：

- 技能结构的技术要求和最佳实践
- 独立技能和增强了 MCP 的工作流模式
- 我们在不同用例中看到行之有效的模式
- 如何测试、迭代并分发你的技能

适用对象：

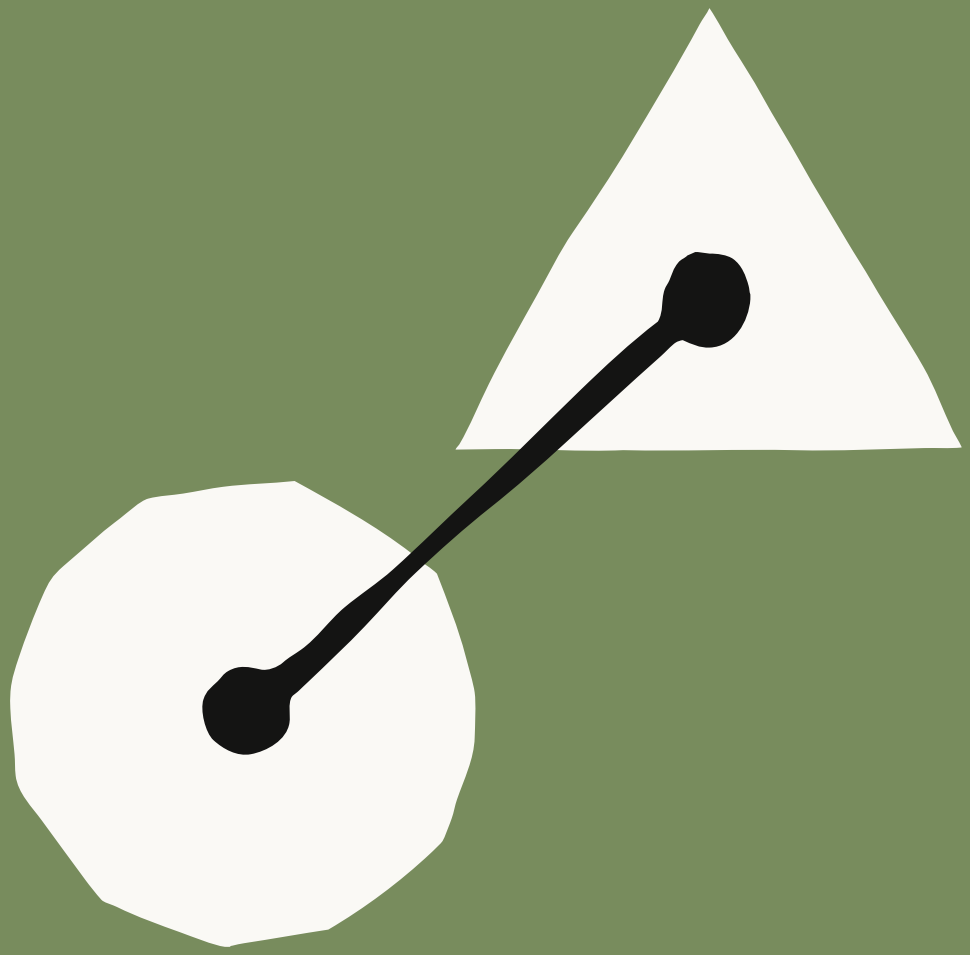
- 希望 Claude 按特定工作流程持续执行的开发者
- 希望 Claude 遵循特定工作流程的高级用户
- 希望在其组织内部标准化 Claude 工作方式的团队

本指南的两条路径

构建独立技能？请侧重基础、规划与设计，以及类别 1-2。要增强 MCP 集成？“Skills + MCP” 一节和类别 3 适合你。两条路径共享相同的技术要求，但你可根据用例选择相关内容。

本指南能带给你的：到最后，你将能够在一次会话内构建出一个可用的技能。使用技能创建器构建并测试你的第一个可运行技能，预计大约需要 15–30 分钟。

我们开始吧。



第一章

基础知识

第1章

基础

什么是技能?

技能是一个包含以下内容的文件夹:

- SKILL.md（必需）：带有 YAML 页眉的 Markdown 格式说明
- scripts/（可选）：可执行代码（Python、Bash 等）
- references/ (可选)：按需加载的文档
- assets/ (可选)：输出中使用的模板、字体、图标

核心设计原则

渐进披露

技能使用三级体系:

- 第一层（YAML frontmatter）：始终加载到 Claude 的系统提示中。提供足够的信息让 Claude 知道何时应使用各个技能，而无需将所有内容加载到上下文中。
- 第二级（SKILL.md 正文）：当 Claude 认为该技能与当前任务相关时加载。包含完整的指令和指南。
- 第三级（关联文件）：打包在技能目录内的附加文件
  这些文件只有在 Claude 需要时才会选择浏览和发现。

这种渐进式披露在保持专业专长的同时，将令牌使用量降到最低。

可组合性

Claude 可以同时加载多个技能。你的技能应能与其他技能良好协作，而不应假设自己是唯一可用的能力。

可移植性

技能在 Claude.ai、Claude Code 和 API 上的工作方式相同。只需创建一次技能，便可在所有平台上使用，无需修改，前提是环境支持该技能所需的任何依赖项。

For MCP Builders: Skills + Connectors

💡 要单独构建不依赖 MCP 的 skills 吗？跳到“规划与设计”部分 —— 你以后随时可以回到这里。

如果你已经有一个可用的 MCP 服务器，那就已经完成了最困难的部分。Skills 是在其之上的知识层——将你已有的工作流程和最佳实践记录下来，使 Claude 能够持续一致地应用这些方法。

厨房类比

MCP 提供了专业厨房：可使用的工具、原料和设备。

技能提供了配方：逐步说明如何创建有价值的事物。

它们共同使用户能够完成复杂任务，而无需自行推敲每一个步骤。

它们如何协同工作：

MCP（连接）	技能（知识）
将 Claude 连接到你的服务（Notion、Asana、Linear 等）	教会 Claude 如何有效使用你的服务
提供实时数据访问和工具调用	捕捉工作流程和最佳实践
Claude 能做的事情	Claude 应该如何去做

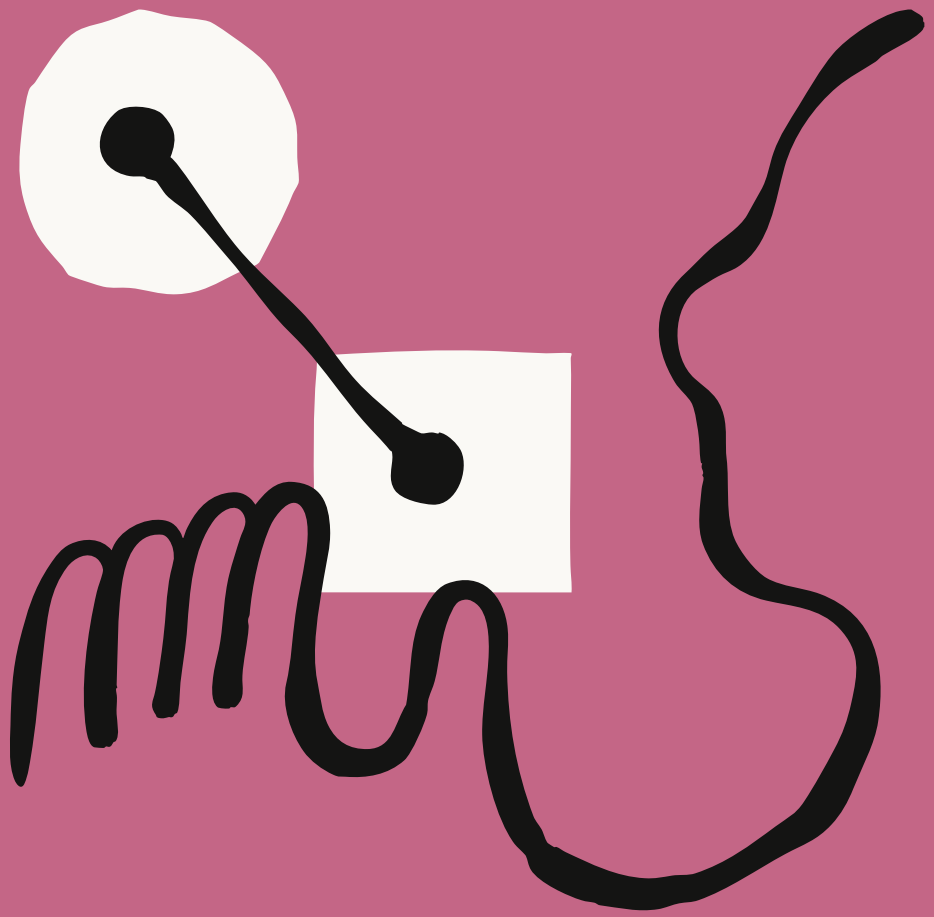
这对您的 MCP 用户为何重要

没有技能：

- 用户连接了你的 MCP，但不知道下一步该做什么
- 支持工单问“我该如何用你们的集成来做 X？”
- 每次对话都从头开始
- 结果不一致，因为用户每次的提示方式不同
- 当真正的问题是工作流程引导时，用户却责怪你的连接器

使用技能后：

- 预构建的工作流在需要时会自动激活
- 工具使用一致且可靠
- 每次交互中都嵌入了最佳实践
- 降低您集成的学习难度



第2章

规划与设计

第2章

规划与设计

以用例为起点

在编写任何代码之前，先确定你的技能应支持的2-3个具体用例。

良好的用例定义：

```
Use Case: Project Sprint Planning
Trigger: User says "help me plan this sprint" or "create
sprint tasks"
Steps:
1. Fetch current project status from Linear (via MCP)
2. Analyze team velocity and capacity
3. Suggest task prioritization
4. Create tasks in Linear with proper labels and estimates
Result: Fully planned sprint with tasks created
```

问你自己：

- 用户想要达成什么目标?
- 这需要哪些多步骤的工作流?
- 需要哪些工具（内置或 MCP）？
- 应嵌入哪些领域知识或最佳实践?

常见技能使用场景类别

在 Anthropic，我们观察到三种常见的用例：

类别 1：文档与资源创建

用途：创建一致且高质量的输出，包括文档、演示文稿、应用、设计、代码等。

真实示例：frontend-design 技能（另请参见 docx、pptx、xlsx 和 ppt 的技能）

“创建具有高设计质量的、可投入生产的独特前端界面。用于构建 Web 组件、页面、产品、海报或应用时。”

关键技术：

- 嵌入的风格指南和品牌标准
- 用于保持输出一致的模板结构
- 定稿前的质量检查清单
- 无需外部工具 —— 使用Claude的内置功能

分类 2：工作流自动化

用于：适合通过一致方法执行的多步骤流程，包括跨多个 MCP 服务器的协调。

真实示例：skill-creator 技能

“用于创建新技能的互动指南。引导用户完成用例定义、frontmatter 生成、说明撰写和验证。”

关键技术：

- 逐步工作流并带有验证关卡
- 常见结构的模板
- 内置的审查与改进建议
- 迭代精化循环

类别 3：MCP 增强

用途：用于工作流指导，以增强对 MCP 服务器提供的工具访问。

真实示例：sentry-code-review 技能（来自 Sentry）

“使用 Sentry 的错误监控数据通过其 MCP 服务器自动分析并修复在 GitHub 拉取请求中检测到的错误。”

关键技术：

- 按顺序协调多个 MCP 调用
- 嵌入领域专业知识
- 提供用户 otherwise 需要指定的上下文
- 针对常见 MCP 问题的错误处理

定义成功标准

你如何判断你的技能在正常工作?

这些是理想目标——粗略的基准而非精确的界限。追求严谨，但要接受在某种程度上会基于直觉的评估。我们正在积极开发更稳健的衡量指南和工具。

量化指标:

- 技能在90%的相关查询中被触发
  – 如何测量：运行10–20个应该触发你技能的测试查询。跟踪它自动加载的次数与需要明确调用的次数。
- 在X次工具调用中完成工作流程
  – 如何测量：比较启用与未启用该技能时完成相同任务的情况。统计工具调用次数和消耗的总令牌数。
- 0 每个工作流的失败 API 调用次数
  – 如何衡量：在测试运行期间监控 MCP 服务器日志。跟踪重试率和错误代码。

定性指标:

- 用户无需提示 Claude 下一步操作
  – 如何评估：在测试期间记录需要重定向或澄清的频率。向内部测试用户或测试用户征求反馈。
- 工作流在无需用户更正的情况下完成
  – 如何评估：对同一请求运行 3–5 次。比较输出以判断结构一致性和质量。
- 在会话间结果一致
  – 如何评估：新用户在最少指导下能否首次尝试就完成任务?

技术要求

文件结构

```
your-skill-name/
├── SKILL.md              # Required - main skill file
├── scripts/              # Optional - executable code
│   ├── process_data.py # Example
│   └── validate.sh # Example
├── references/           # Optional - documentation
│   ├── api-guide.md # Example
│   └── examples/ # Example
└── assets/              # Optional - templates, etc.
    └── report-template.md # Example
```

关键规则

SKILL.md 命名：

- 必须完全为 `SKILL.md` (区分大小写)
- 不接受任何变体（SKILL.MD、skill.md 等）

技能文件夹命名：

- 使用 kebab-case：`notion-project-setup` ✅
- 不允许空格：`Notion Project Setup` ❌
- 不允许下划线：`notion_project_setup` ❌
- 不使用大写字母：`NotionProjectSetup` ❌

不要 README.md：

- 不要将 README.md 放在你的 skill 文件夹内
- 所有文档应放在 SKILL.md 或 references/ 中
- 注意：通过 GitHub 分发时，你仍然需要一个仓库级别的 README 供人类用户阅读 — 请参阅 分发与共享。

YAML frontmatter：最重要的部分

YAML 前置元数据决定是否加载你的 skill。请确保其正确无误。

最小所需格式

```
---
name: your-skill-name
description: What it does. Use when user asks to [specific
phrases].
---
```

这就是你开始所需的一切。

字段要求

name（必填）：

- 仅 kebab-case
- 不要空格或大写字母
- 应与文件夹名称匹配

描述（必填）：

- 必须同时包含两项：
  - 技能的功能
  - 何时使用（触发条件）
- 少于 1024 字符
- 无 XML 标签（< 或 >）
- 包含用户可能提出的具体任务
- 如相关，请注明文件类型

许可证（可选）：

- 如果要将技能开源请使用
- 常见：MIT、Apache-2.0

兼容性（可选）

- 1-500 个字符
- 指明环境要求：例如目标产品、所需系统包、网络访问需求等。

元数据（可选）：

- 任何自定义键值对
- 建议：author、version、mcp-server
- 示例：
  ```yaml
  metadata:
      author: ProjectHub
      version: 1.0.0 mcp-server: projecthub
  ```

安全限制

前置内容中禁止：

- XML 尖括号（< >）
- 名称中包含 "claude" 或 "anthropic" 的技能（保留）

原因：Frontmatter 出现在 Claude 的系统提示中。恶意内容可能会注入指令。

编写高效技能

描述字段

根据 Anthropic 的工程博客："这些元数据……提供了足够的信息让 Claude 知道何时应使用每项技能，而无需将所有内容都加载到上下文中。" 这是渐进式披露的第一级。

结构：

```
[What it does] + [When to use it] + [Key capabilities]
```

优秀描述示例：

```
# Good - specific and actionable
description: Analyzes Figma design files and generates
developer handoff documentation. Use when user uploads .figfiles,
 asks for "design specs", "component documentation", or
"design-to-code handoff".

# Good - includes trigger phrases
description: Manages Linear project workflows including sprint
planning, task creation, and status tracking. Use when user
mentions "sprint", "Linear tasks", "project planning",
 or asksto "create tickets".

# Good - clear value proposition
description: End-to-end customer onboarding workflow for
PayFlow. Handles account creation, payment setup, and
subscription management. Use when user says "onboard newcustomer",
 "set up subscription", or "create PayFlow account".
```

糟糕描述的示例：

```
# Too vague
description: Helps with projects.

# Missing triggers
description: Creates sophisticated multi-page documentation
systems.

# Too technical, no user triggers
description: Implements the Project entity model with
hierarchical relationships.
```

撰写主要说明

在 frontmatter 之后，以 Markdown 撰写实际说明。

推荐结构：

将此模板调整为适合您的技能。将方括号部分替换为您的具体内容。

```
---
name: your-skill
description: [...]
---

# Your Skill Name

## Instructions

### Step 1: [First Major Step]
Clear explanation of what happens.

Example:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Expected output: [describe what success looks like]
```
```

（根据需要添加更多步骤）

示例

示例 1：[常见情形]

用户说：“设置一个新的营销活动”

操作：
1. 通过 MCP 获取现有活动
2. 使用提供的参数创建新活动

结果：活动已创建并附确认链接

（根据需要添加更多示例）

疑难解答

错误：[常见错误信息]

原因：[发生原因]

解决方案：[如何修复]

（根据需要添加更多错误情况）

编写指令的最佳实践

具体且可执行

✅ 良好示例:

```
Run `python scripts/validate.py --input {filename}` to check
data format.
If validation fails, common issues include:
- Missing required fields (add them to the CSV)
- Invalid date formats (use YYYY-MM-DD)
```

❌ 不良示例:

```
Validate the data before proceeding.
```

包含错误处理

```
## Common Issues

### MCP Connection Failed
If you see "Connection refused":
1. Verify MCP server is running: Check Settings > Extensions
2. Confirm API key is valid
3. Try reconnecting: Settings > Extensions > [Your Service] >
Reconnect
```

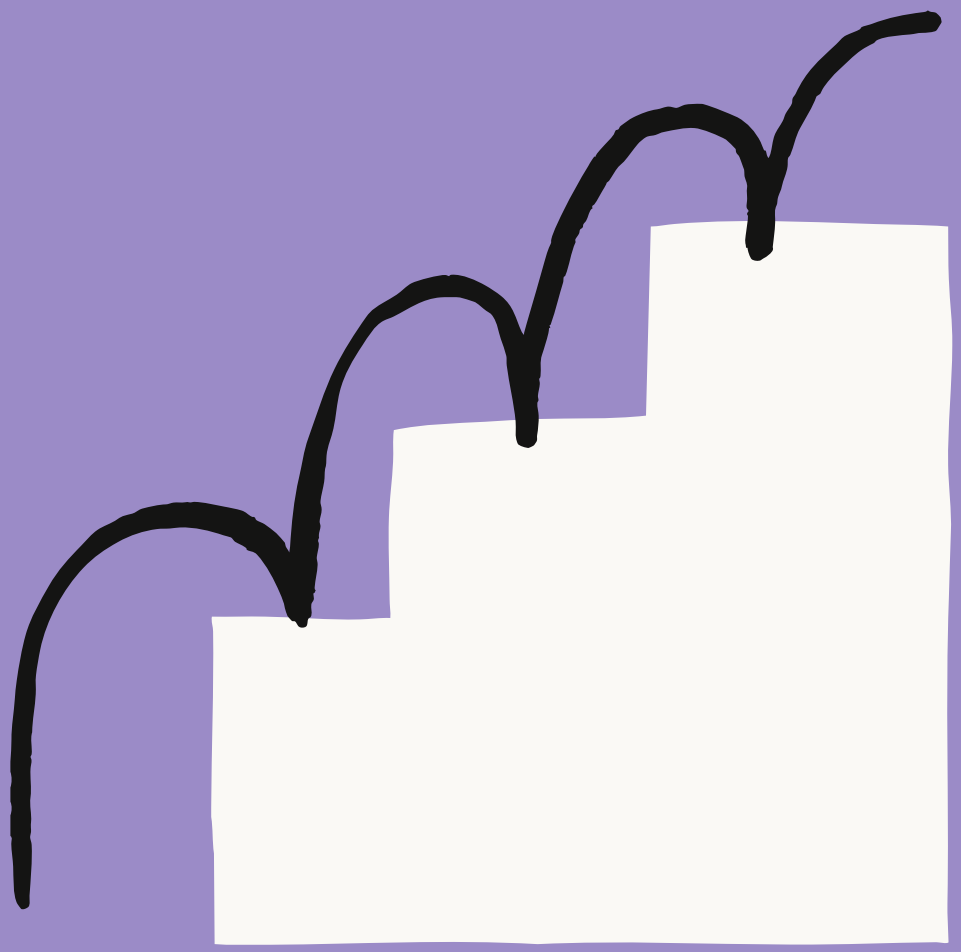
清晰地引用捆绑资源

```
Before writing queries, consult `references/api-patterns.md`for:

- Rate limiting guidance
- Pagination patterns
- Error codes and handling
```

使用渐进式披露

将 SKILL.md 保持在核心说明上。将详细文档移至`references/`并链接到它。（参见“三层”系统如何工作的核心设计原则。）



第3章

测试与迭代

第3章

测试与迭代

可以根据需求以不同严格程度来测试技能：

- 在 Claude.ai 中手动测试——直接运行查询并观察行为。迭代快速，无需设置。
- 在 Claude Code 中的脚本化测试——为可重复验证自动化测试用例，以便在变更后进行验证。
- 通过 skills API 进行编程式测试——构建评估套件，按定义的测试集系统性运行。

选择与您的质量要求和技能可见性相匹配的方法。供小团队内部使用的技能，其测试需求与部署给数千名企业用户的技能不同。

专业提示：在扩展之前先对单个任务进行迭代

我们发现，最有效的技能创建者会在单个具有挑战性的任务上不断迭代，直到 Claude 成功，然后将获胜的方法提取成技能。这利用了 Claude 的上下文学习能力，并比广泛测试提供更快的信号。一旦你有了可行的基础，就扩大到多个测试用例以覆盖更多情况。

推荐的测试方法

根据早期经验，行之有效的技能测试通常涵盖三个方面：

1. 触发测试

目标：确保你的技能在正确的时机加载。

测试用例：

- ✅ 在明显的任务上触发
- ✅ 触发于意译后的请求
- ❌ 不会在不相关的话题上触发

示例测试套件：

```
Should trigger:
- "Help me set up a new ProjectHub workspace"
- "I need to create a project in ProjectHub"
- "Initialize a ProjectHub project for Q4 planning"

Should NOT trigger:
- "What's the weather in San Francisco?"
- "Help me write Python code"
- "Create a spreadsheet" (unless ProjectHub skill handles
sheets)
```

2. 功能测试

目标：验证该技能生成的输出是否正确。

测试用例：

- 生成有效输出
- API 调用成功
- 错误处理正常
- 覆盖边界情况

示例：

```
Test: Create project with 5 tasks
Given: Project name "Q4 Planning", 5 task descriptions
When: Skill executes workflow
Then:
  - Project created in ProjectHub
  - 5 tasks created with correct properties
  - All tasks linked to project
  - No API errors
```

3. 性能比较

目标：证明该技能相比基线能够提升结果。

使用“定义成功标准”中的指标。以下是可能的比较示例。

基线比较：

```
Without skill:
- User provides instructions each time
- 15 back-and-forth messages
- 3 failed API calls requiring retry
- 12,000 tokens consumed
```

```
With skill:
- Automatic workflow execution
- 2 clarifying questions only
- 0 failed API calls
- 6,000 tokens consumed
```

使用 skill-creator 技能

The `skill-creator` skill - 可通过 Claude.ai 的插件目录或下载用于 Claude Code - 帮助你构建并迭代技能。如果你有 MCP 服务器并且知道你最常用的 2–3 个工作流，你可以在一次工作中构建并测试出一个可用的技能——通常只需 15–30 分钟。

创建技能：

- 从自然语言描述生成技能
- 生成带有 frontmatter 的正确格式化 SKILL.md
- 建议触发短语和结构

复审技能：

- 标注常见问题（描述含糊、缺少触发项、结构性问题）
- 识别潜在的过度/不足触发风险
- 根据技能的声明目的建议测试用例

迭代改进：

- 在使用你的技能并遇到边缘情况或失败后，将这些示例反馈给技能创建者
- 示例：“使用本次对话中发现的问题和解决方案来改进该技能处理[特定边缘情况]的方式”

使用方法：

```
"Use the skill-creator skill to help me build a skill for
[your use case]"
```

注意：skill-creator 可以帮助你设计和完善技能，但不会执行自动化测试套件或生成定量评估结果。

基于反馈的迭代

技能是不断演进的文档。请计划基于以下内容进行迭代：

触发不足的信号：

- 技能在应加载时未加载
- 用户手动启用它
- 关于何时使用它的支持问题

> 解决方案：在描述中增加更多细节和细微差别——这可能包括针对技术术语的关键词

过度触发的信号：

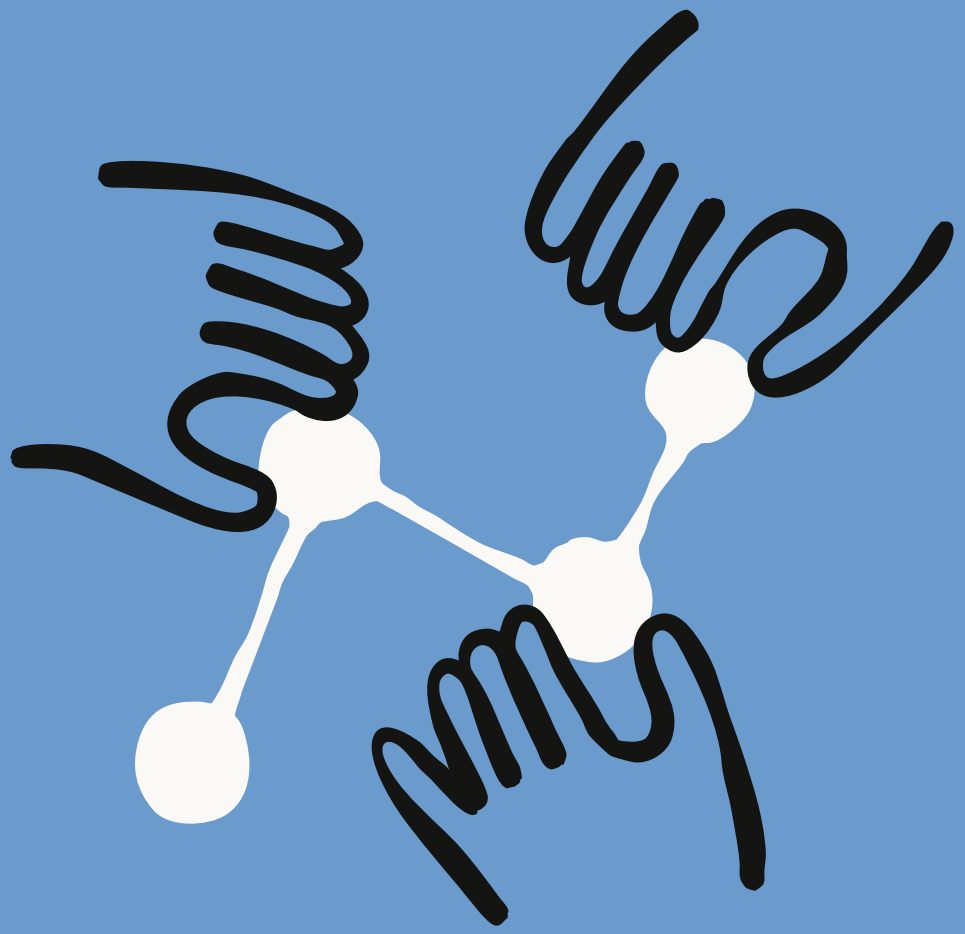
- 针对无关查询的技能加载
- 用户禁用它
- 对目标的困惑

> 解决方案：添加否定触发器，变得更具体

执行问题：

- 结果不一致
- API 调用失败
- 需要用户修正

> 解决方案：改进说明，添加错误处理



第4章

分发与共享

第4章

分发与共享

技能使您的 MCP 集成更加完整。当用户比较连接器时，具备技能的连接器能更快体现价值，使您相比仅有 MCP 的替代方案更具优势。

当前分发模型（2026年1月）

个人用户如何获取技能：

1. 下载技能文件夹
2. 将文件夹压缩（如有需要）
3. 通过 Settings > Capabilities > Skills 上传到 Claude.ai
4. 或放入 Claude Code 的 skills 目录中

组织级别的技能：

- 管理员可以在整个工作区部署技能（已于 2025 年 12 月 18 日发布）
- 自动更新
- 集中式管理

一个开放标准

我们已发布 **Agent Skills** 作为一项开放标准。像 MCP 一样，我们认为技能 应该可以跨工具和平台移植——同一项技能应能在 你使用 Claude 或其他 AI 平台时工作。话虽如此，某些技能旨在充分利用特定平台的功能；作者可以 在技能的 `compatibility` 字段中注明。我们一直在与生态系统成员协作制定该标准，并对早期采用情况感到兴奋。

通过 API 使用技能

对于编程场景——例如构建应用、代理或利用技能的自动化工作流——该 API 提供对技能管理和执行的直接控制。

主要功能：

- `/v1/skills` 用于列出和管理技能的端点
- 通过 `container.skills` 参数向 Messages API 请求添加技能
- 通过 Claude 控制台进行版本控制和管理
- 可与用于构建自定义代理的 Claude Agent SDK 配合使用

何时通过 API 使用技能与在 Claude.ai 使用技能：

使用场景	最佳呈现端
最终用户直接与技能交互	Claude.ai / Claude Code
开发期间的手动测试与迭代	Claude.ai / Claude Code
个体的、临时的工作流程	Claude.ai / Claude Code
以编程方式使用技能的应用程序	API
大规模的生产部署	API
自动化流水线和智能体系统	API

注意：API 中的技能需要 Code Execution Tool 测试版，该测试版为技能运行提供所需的安全环境。

有关实现细节，请参见：

- 技能 API 快速入门
- 创建自定义技能
- Agent SDK 中的技能

当前推荐的方法

首先将您的技能托管在 GitHub 的公共仓库中，附上清晰的 README（供访问者阅读——这与您的技能文件夹不同，技能文件夹不应包含 README.md），并提供带有截图的示例用法。然后在您的 MCP 文档中添加一个章节，链接到该技能，说明将两者结合使用的价值，并提供快速入门指南。

1. 在 GitHub 上托管

- 将技能放在公开仓库以便开源
- 在 README 中清晰说明安装步骤
- 示例用法和截图

2. 在你的 MCP 仓库中编写文档

- 在 MCP 文档中链接到这些技能
- 解释同时使用两者的价值
- 提供快速入门指南

3. 创建安装指南

```
## Installing the [Your Service] skill

1. Download the skill:
   - Clone repo: `git clone https://github.com/yourcompany/
     skills`
   - Or download ZIP from Releases

2. Install in Claude:
   - Open Claude.ai > Settings > skills
   - Click "Upload skill"
   - Select the skill folder (zipped)

3. Enable the skill:
   - Toggle on the [Your Service] skill
   - Ensure your MCP server is connected

4. Test:
   - Ask Claude: "Set up a new project in [Your Service]"
```

定位你的技能

你如何描述你的技能决定了用户是否理解其价值并实际尝试。在撰写有关你的技能的内容时——无论是在 README、文档还是营销材料中——请牢记以下原则。

关注结果，而非功能：

✅ 良好：

```
"The ProjectHub skill enables teams to set up complete project
workspaces in seconds — including pages, databases, and
templates — instead of spending 30 minutes on manual setup."
```

❌ 不良：

```
"The ProjectHub skill is a folder containing YAML frontmatter
and Markdown instructions that calls our MCP server tools."
```

突出显示 MCP + 技能 故事：

```
"Our MCP server gives Claude access to your Linear projects.
Our skills teach Claude your team's sprint planning workflow.
Together, they enable AI-powered project management."
```

第5章

模式与故障排除

第5章

模式与故障排除

这些模式来自早期采用者和内部团队创建的技能。它们代表了我们观察到的一些常见且运行良好的方法，而不是强制性的模板。

选择方法：以问题为先 vs. 以工具为先

把它想象成家得宝。你可能带着一个问题走进来——“我需要修理一个厨房橱柜”——店员会把合适的工具指给你。或者你可能先挑了一把新电钻，再询问如何用它去完成具体工作。

技能的运作方式也是如此：

- 以问题为先：“我需要建立一个项目工作区” → 你的技能会以正确的顺序协调合适的 MCP 调用。用户描述期望结果；技能负责处理工具。

- 以工具为先：“我已连接 Notion MCP” → 你的技能向 Claude 传授最佳工作流程和最佳实践。用户已有访问权限；技能提供专业知识。

大多数技能偏向于某一种方向。知道哪种框架适合你的使用场景有助于你在下面选择合适的模式。

模式 1：顺序工作流编排

使用场景：当你的用户需要按特定顺序执行多步流程时。

示例结构：

```
## Workflow: Onboard New Customer

### Step 1: Create Account
Call MCP tool: `create_customer`
Parameters: name, email, company

### Step 2: Setup Payment
Call MCP tool: `setup_payment_method`
Wait for: payment method verification

### Step 3: Create Subscription
Call MCP tool: `create_subscription`
Parameters: plan_id, customer_id (from Step 1)

### Step 4: Send Welcome Email
Call MCP tool: `send_email`
Template: welcome_email_template
```

关键技术：

- 显式步骤顺序
- 步骤之间的依赖关系
- 各阶段的验证
- 失败时的回滚指令

模式 2：多 MCP 协同

适用场景：工作流跨越多个服务时使用。

示例：从设计到开发的交接

```
### Phase 1: Design Export (Figma MCP)
1. Export design assets from Figma
2. Generate design specifications
3. Create asset manifest

### Phase 2: Asset Storage (Drive MCP)
1. Create project folder in Drive
2. Upload all assets
3. Generate shareable links

### Phase 3: Task Creation (Linear MCP)
1. Create development tasks
2. Attach asset links to tasks
3. Assign to engineering team

### Phase 4: Notification (Slack MCP)
1. Post handoff summary to #engineering
2. Include asset links and task references
```

关键技术：

- 明确的阶段分离
- MCP 之间的数据传递
- 在进入下一阶段前进行验证
- 集中式错误处理

模式 3：迭代优化

适用情况：输出质量会随着迭代而提升。

示例：报告生成

```
## Iterative Report Creation

### Initial Draft
1. Fetch data via MCP
2. Generate first draft report
3. Save to temporary file

### Quality Check
1. Run validation script: `scripts/check_report.py`
2. Identify issues:
   - Missing sections
   - Inconsistent formatting
   - Data validation errors

### Refinement Loop
1. Address each identified issue
2. Regenerate affected sections
3. Re-validate
4. Repeat until quality threshold met

### Finalization
1. Apply final formatting
2. Generate summary
3. Save final version
```

关键技术：

- 明确的质量标准
- 迭代改进
- 验证脚本
- 知道何时停止迭代

模式 4：上下文感知的工具选择

使用场景：在相同目标下，根据上下文选择不同工具。

示例：文件存储

```
## Smart File Storage

### Decision Tree
1. Check file type and size
2. Determine best storage location:
   - Large files (>10MB): Use cloud storage MCP
   - Collaborative docs: Use Notion/Docs MCP
   - Code files: Use GitHub MCP
   - Temporary files: Use local storage

### Execute Storage
Based on decision:
- Call appropriate MCP tool
- Apply service-specific metadata
- Generate access link

### Provide Context to User
Explain why that storage was chosen
```

关键技术：

- 明确的决策标准
- 备用方案
- 关于选择的透明度

模式 5：领域特定智能

使用场景：当你的技能在工具访问之外还提供了专门知识时使用。

示例：金融合规

```
## Payment Processing with Compliance

### Before Processing (Compliance Check)
1. Fetch transaction details via MCP
2. Apply compliance rules:
   - Check sanctions lists
   - Verify jurisdiction allowances
   - Assess risk level
3. Document compliance decision

### Processing
IF compliance passed:
  - Call payment processing MCP tool
  - Apply appropriate fraud checks
  - Process transaction
ELSE:
  - Flag for review
  - Create compliance case

### Audit Trail
- Log all compliance checks
- Record processing decisions
- Generate audit report
```

关键技术：

- 将领域专业知识嵌入逻辑中
- 在行动前确保合规
- 全面的文档
- 明确的治理

故障排除

技能无法上传

错误："在上传的文件夹中找不到 SKILL.md"

原因：文件未精确命名为 SKILL.md

解决方案：

- 重命名为 SKILL.md（区分大小写）
- 通过以下命令验证：ls -la 应显示 SKILL.md

错误：“无效的 frontmatter”

原因：YAML 格式问题

常见错误：

```
# Wrong - missing delimiters
name: my-skill
description: Does things

# Wrong - unclosed quotes
name: my-skill
description: "Does things

# Correct
---
name: my-skill
description: Does things
---
```

错误：“无效的技能名称”

原因：名称包含空格或大写字母

```
# Wrong
name: My Cool Skill

# Correct
name: my-cool-skill
```

技能未触发

症状：技能从未自动加载

Fix:

修改你的描述字段。参见「描述字段」以获取好的/不好的示例。

快速检查清单：

- 是否过于笼统？（“有助于项目”不行）
- 是否包含用户实际会说的触发短语？
- 如果适用，是否提到相关的文件类型？

调试方法：

询问 Claude："你会在什么时候使用 [skill name] skill？" Claude 会引用该描述并返回。根据缺失的部分进行调整。

技能触发过于频繁

症状：技能在不相关的查询中被加载

解决方案：

1. 添加负面触发器

```
description: Advanced data analysis for CSV files. Use for
statistical modeling, regression, clustering. Do NOT use for
simple data exploration (use data-viz skill instead).
```

2. 更具体

```
# Too broad
description: Processes documents

# More specific
description: Processes PDF legal documents for contract review
```

3. 澄清范围

```
description: PayFlow payment processing for e-commerce. Use
specifically for online payment workflows, not for general
financial queries.
```

MCP 连接问题

症状：技能加载但 MCP 调用失败

检查清单：

1. 验证 MCP 服务器已连接
   – Claude.ai：设置 > 扩展 > [您的服务]
   – 应显示“已连接”状态
2. 检查认证
   – API 密钥有效且未过期
   – 已授予适当的权限/作用域
   – OAuth 令牌已刷新
3. 独立测试 MCP
   – 让 Claude 直接调用 MCP（不通过 skill）
   – "使用 [Service] MCP 来获取我的项目"
   – 如果这失败，则问题出在 MCP 而非 skill
4. 验证工具名称
   – skill 引用的是正确的 MCP 工具名称
   – 检查 MCP 服务器文档
   – 工具名称区分大小写

未遵循说明

症状：技能已加载，但 Claude 未按指示操作

常见原因：

1. 说明过于冗长
   – 保持说明简洁
   – 使用项目符号和编号列表
   – 将详细参考移至单独文件
2. 指示被埋没
   – 将关键指示放在顶部
   – 使用 ## Important 或 ## Critical 标题
   – 如有必要重复要点
3. 语言含糊不清

```
# Bad
Make sure to validate things properly

# Good
CRITICAL: Before calling create_project, verify:
- Project name is non-empty
- At least one team member assigned
- Start date is not in the past
```

高级技术：对于关键性验证，建议打包一个以编程方式执行检查的脚本，而不是依赖语言指令。代码是确定性的；语言解释则不是。参见 Office 技能，了解此模式的示例。

4. 模型“懒惰” 添加明确的鼓励：

```
## Performance Notes
- Take your time to do this thoroughly
- Quality is more important than speed
- Do not skip validation steps
```

注意：将此添加到用户提示中比写在 SKILL.md 中更有效

大上下文问题

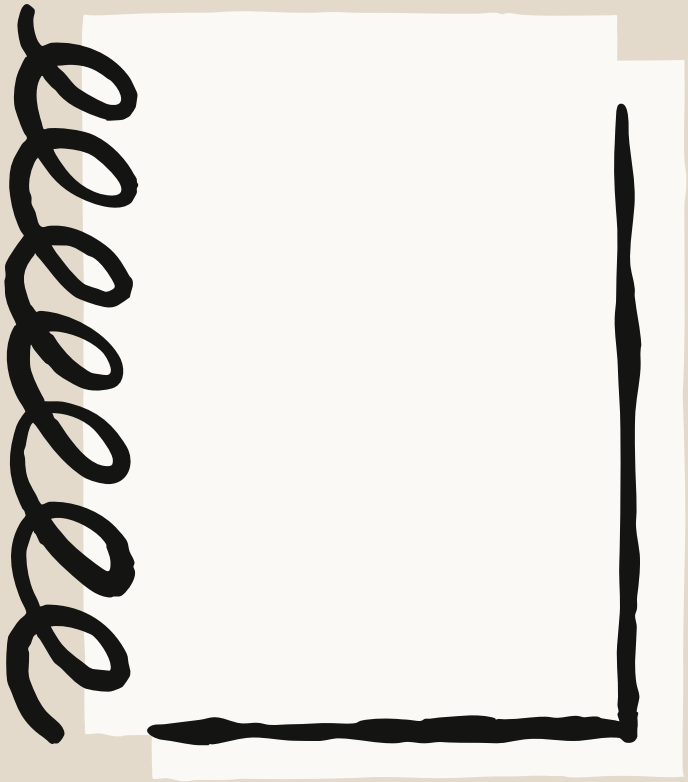
症状：技能似乎变慢或响应性能下降

原因：

- 技能内容过大
- 同时启用的技能过多
- 一次性加载所有内容而非逐步展示

解决方案：

1. 优化 SKILL.md 大小
   - 将详细文档移至 references/ 目录
   - 使用链接指向 references 而不是内联显示
   - 保持 SKILL.md 在 5,000 字以内
2. 减少启用的技能数量
   - 评估是否同时启用了超过 20 到 50 个技能
   - 建议有选择地启用
   - 考虑为相关功能提供技能“包”



第6章

资源与参考资料

第6章

资源与参考

如果您正在构建第一个技能，请从《最佳实践指南》开始，然后根据需要参考 API 文档。

官方文档

Anthropic 资源：

- 最佳实践指南
- 技能文档
- API 参考
- MCP 文档

博客文章：

- 介绍代理技能
- 工程博客：为现实世界装备代理
- 技能说明
- 如何为 Claude 创建技能
- 为 Claude Code 构建技能
- 通过技能改进前端设计

示例技能

公共技能代码仓库：

- GitHub：anthropics/skills
- 包含由 Anthropic 创建、可供你自定义的技能

工具与实用程序

skill-creator 技能：

- 内置于 Claude.ai 并可用于 Claude Code
- 可根据描述生成技能
- 审查并提供建议
- 用法："帮我用 skill-creator 构建一个技能"

验证：

- skill-creator 可以评估你的技能
- 询问："审查此技能并提出改进建议"

获取支持

有关技术问题：

- 常见问题：Claude 开发者 Discord 社区论坛

有关错误报告：

- GitHub 问题：anthropics/skills/issues
- 请包括：技能名称、错误信息、重现步骤

参考 A：快速 checklist

在上传前后使用此清单来验证您的技能。 如果您想更快速地开始，请使用 skill-creator 技能生成您的初稿，然后运行此清单以确保您没有遗漏任何内容。

开始之前

- ☐ 确定了 2-3 个具体使用场景
- ☐ 已确认工具（内置或 MCP）
- ☐ 已查看本指南和示例技能
- ☐ 已规划文件夹结构

开发中

- ☐ 文件夹名称使用 kebab-case
- ☐ 存在 SKILL.md 文件（拼写完全相同）
- ☐ YAML 前置信息使用 --- 分隔符
- ☐ name 字段：kebab-case，不能有空格，不能有大写字母
- ☐ description 包含 WHAT 和 WHEN
- ☐ 任何地方都不能有 XML 标签（< >）
- ☐ 说明要清晰且可操作
- ☐ 包含错误处理
- ☐ 提供示例
- ☐ 参考资料已清楚链接

上传前

- ☐ 在明显任务上测试触发情况
- ☐ 在意译请求上测试触发情况
- ☐ “已验证”不会在无关话题上触发
- ☐ 功能测试通过
- ☐ 工具集成正常工作（如适用）
- ☐ 已压缩为 .zip 文件

上传后

- ☐ 在真实对话中测试
- ☐ 监控触发过少或过多
- ☐ 收集用户反馈
- ☐ 针对描述和指令进行迭代
- ☐ 在元数据中更新版本

参考 B：YAML 文件头

必填字段

```
---
name: skill-name-in-kebab-case
description: What it does and when to use it. Include specific
trigger phrases.
---
```

所有可选字段

```
name: skill-name
description: [required description]
license: MIT # Optional: License for open-source
allowed-tools: "Bash(python:*) Bash(npm:*) WebFetch" # Optional:
Restrict tool access
metadata: # Optional: Custom fields
  author: Company Name
  version: 1.0.0
  mcp-server: server-name
  category: productivity
  tags: [project-management, automation]
  documentation: https://example.com/docs
  support: support@example.com
```

安全注意事项

允许：

- 任何标准 YAML 类型（字符串、数字、布尔值、列表、对象）
- 自定义元数据字段
- 长描述（最多 1024 字符）

禁止：

- XML 尖括号（< >）——安全限制
- YAML 中的代码执行（使用安全 YAML 解析）
- 以“claude”或“anthropic”为前缀命名的技能（保留）

参考 C：完整技能示例

欲了解展示本指南模式的完整可投入生产环境的技能：

- 文档技能 - 创建 PDF、DOCX、PPTX、XLSX
- 示例技能 - 各种工作流模式
- 合作伙伴技能目录 - 查看来自 Asana 等各方合作伙伴的技能，Atlassian、Canva、Figma、Sentry、Zapier 等更多

这些代码仓库保持最新状态，并包含比此处覆盖内容更多的示例。克隆它们，根据你的用例进行修改，并将它们用作模板。

Claude

claude.ai